TD1 : Compteurs de passages de vélos Rennes Métropole

Préambule

1. Créer sur votre disque un dossier PythonOpenData dans lequel vous réaliserez l'ensemble des travaux de cette matière.
2. Dans votre dossier PythonOpenData, créez un sous-répertoire TD1.
3. Lancer l'éditeur Visual Studio Code.
4. Dans Visual Studio Code, ouvrir le dossier PythonOpenData/TD1.

Ce TD se concentre sur un jeu de données de comptage du nombre de passages de vélos à Rennes au niveau d'un certain nombres de bornes de comptage.

Exercice 1 : Etude des données

Ce jeu de données est issu de la plateforme Open Data de Rennes Métropole, et l'on s'est restreint ici au mois d'avril 2026. Les données sont fournies au format JSON (JavaScript Object Notation). Ce format permet de stocker des données structurées, par exemple avec une organisation hiérarchique.

1. Sur CURSUS, télécharger le jeu de données `counter_avril2026.json` et l'enregistrer dans le dossier PythonOpenData/TD1.
2. Visualiser avec Visual Studio Code le contenu du fichier JSON.
3. Répondre - en groupe - aux questions suivantes :
   — Quelle est la nature python du contenu du json ?
   — Quels sont les attributs (clés de dictionnaires) de ce jeu de données ?
   — Que contient l'attribut `date` ?
   — Donner quelques uns des noms de compteurs (attribut `name`).

Dans la suite de ce sujet, on nomme « enregistrement » chaque dictionnaire issu du fichier JSON. Notons que chaque enregistrement contient une date. Ainsi, une date codée sous la forme :

```
"date": {
    "year": 2026,
    "month": 4,
    "day": 2,
    "hour": 10
}
```

correspond au créneau `10h-11h` le 2 avril 2026 et l'enregistrement correspondant indique donc le nombre de passages de vélos au niveau de la borne de comptage considérée sur ce créneau.

Exercice 2 : Extraction d'informations élémentaires

Les questions suivantes nécessitent de manipuler les données depuis Python et le code produit devra être inclus dans le fichier `td1.py`.

1. Charger le contenu du fichier `counter_avril2026.json` dans une variable `donnees_2026`. Cette variable doit contenir une liste des enregistrements, chaque enregistrement étant stocké dans un dictionnaire.
2. Afficher le nombre d'enregistrements contenu dans le fichier.
3. Afficher le 1er enregistrement.
4. Écrire une fonction `nom_compteurs` qui prend en entree la liste des enregistrements et retourne une liste sans doublon de tous les noms de compteurs (attribut `name`) contenus dans le jeu de données.

Exercice 3 : Comptages de vélos par compteur

Problèmes posés : Quel compteur a le plus de passages sur un mois ? A quelle heure la fréquentation est-elle la plus forte ?

1. Écrire une fonction `compte_passages_compteurs` qui prend en entrée le jeu de données et la liste des noms de compteur. Cette fonction renvoie un dictionnaire dont les clés sont les noms de compteurs et les valeurs le nombre total de passages de vélos sur ce compteur.
2. Écrire une fonction `compte_passage_horaire` qui prend en entrée le jeu de données et retourne, le nombre de passages de vélo pour chaque heure de la journée (0h-1h, 1h-2h, ..., 23h-24h), tous compteurs et tous jours confondus. Le résultat sera renvoyé sous la forme d'un dictionnaire dont les clés sont les horaires, et les valeurs le nombre total de passages.
3. En cherchant dans ce dictionnaire, afficher l'heure du jour à laquelle on compte le plus grand nombre de passages de vélos, tous compteurs et tous jours confondus.

Exercice 4 : Comparaison 2024 - 2026

On donne un second fichier, qui donne les valeurs similaires, pour le mois d'avril 2024.

1. Sur CURSUS, télécharger le jeu de données `counter_avril2024.json` et l'enregistrer dans le dossier PythonOpenData/TD1.
2. Charger le contenu du fichier `counter_avril2024.json` dans une variable `donnees_2024`.
3. Écrire une fonction `compte_passages_jours` qui prend en paramètre un jeu de données ainsi qu'un nombre représentant le jour dans le mois. Cette fonction renvoie le nombre de passages pour ce jour, tous horaires et tous compteurs confondus.
4. Tester cette fonction pour afficher le nombre de passages le 9 avril en 2024 et en 2026.
5. Écrire une fonction `compare_tous_les_jours` qui prend en paramètre les données 2024 et les données 2026, et produit un dictionnaire contenant les données comparatives pour tous les jours d'avril, au format ci-dessous. *On pourra de façon simple utiliser la fonction de la question 3, ou bien envisager une autre méthode avec une combinatoire plus réduite !*

   `{1: {2024: 1374, 2026: 4193}, 2: {2024: 3481, 2026: 4557}, 3: {2024: 3910, 2026: 3677}...`

Exercice 5 : Export au format JSON

Dans cette partie, il va s'agir d'enregistrer un export de la comparaison 2024-2026, jour par jour.

1. Écrire une fonction `exporte_comparaison` qui prend en entrée les 2 jeux de données de 2024 et 2026, et écrit dans un fichier nommé `nb_passages_avril-2024_2026.json` le dictionnaire renvoyé par la fonction `compare_tous_les_jours`.
2. Pour une meilleure lisibilité, vous ferez en sorte de fixer l'indentation du fichier obtenu à 2 caractères.